

Concurrent Localisation and Map Building for Mobile Robots Using Ultrasonic Sensors

W.D. Rencken

Siemens AG, Corporate Research and Development

Otto-Hahn-Ring 6

81739 Munich

Germany

e-mail: rencken@zfe.siemens.de

Abstract

Successful autonomous navigation by a mobile robot depends on its ability to map its environment and to accurately determine its position within this environment. In complex environments where no additional navigational aids are present, the localisation and map building processes are not independent of each other.

This paper presents an approach where the bootstrapping problem of concurrent localisation and map building is solved by estimating the respective errors introduced by each of the processes and correcting them accordingly. The algorithms were tested in simulation and results showed that the robot stably navigated in its environment.

1 Introduction

Localisation and map building are two key components for the autonomous operation of mobile robots. The localisation process enables the robot to determine its position within its environment, based upon observed landmarks. These landmarks can either exist in the form of fixed beacons, or regular features within the environment, such as planes, corners etc. The set of landmarks, defined as the robot's map, can either be known beforehand or it has to be built up during the robot's operation. Building the robot's map however requires that the robot's position be known accurately so that map consistency is ensured. Hence the robot is faced with the classic bootstrapping problem. Due to this dilemma, most work has concentrated exclusively either on localisation [4] [2] [5] or map building [9] [3]. This implies that the robot is either supplied with an a priori map of its environment or its position is known accurately.

This work addresses the problem of when the localisation and map building processes cannot be decoupled completely. As a result no fixed navigation beacons exist in the environment and no perfect position information is available.

1.1 Definition of the General Problem

Using the notation introduced by Leonard et al [6], the robot's map is defined as follows

$$M = \{p_i | 1 \leq i \leq n\} \quad (1)$$

where p_i is a geometric feature such as a plane or corner, and n is the number of features within the environment.

The state of the vehicle at time k is defined as

$$x(k) = [x(k), y(k), \theta(k)]^T \quad (2)$$

where $x(k)$ and $y(k)$ are the absolute position of the robot in world coordinates and $\theta(k)$ is its orientation with respect to the x-axis of the coordinate frame.

The map and the robot's state together form the system state vector

$$x_s(k) = [x(k), M]^T \quad (3)$$

which has to be estimated by the robot. Since perfect knowledge is not available, the estimated state vector $\hat{x}_s(k|k)$ has uncertainty associated with it in the form of a covariance matrix $\Lambda(k|k)$

$$\Lambda(k|k) = \begin{bmatrix} P(k|k) & C_{R1}(k|k) & \cdots & C_{Rn}(k|k) \\ C_{R1}(k|k) & \Lambda_1(k|k) & \cdots & C_{1n}(k|k) \\ \vdots & \vdots & \ddots & \vdots \\ C_{Rn}(k|k) & C_{1n}(k|k) & \cdots & \Lambda_n(k|k) \end{bmatrix} \quad (4)$$

where $C_{Ri}(k|k)$ is the vehicle to target cross-covariance, $C_{ij}(k|k)$ is the target to target cross-covariance, $P(k|k)$ is the vehicle's covariance and Λ_i is the target's covariance.

The robot is only able to estimate the system state by associating a set of sensor observations at time k

$$Z(k) = \{z_j | 1 \leq j \leq m(k)\} \quad (5)$$

where $m(k)$ is the number of sensor observations, to the map M .

1.2 Previous Work

The first approach that describes the simultaneous localisation and map building process was presented by Smith et al [8]. They use an extended kalman filter (EKF) to estimate the system state and assume full knowledge of all the covariance and cross-covariance matrices as well as perfect data-to-target association. The approach uses two steps. First the position of the robot is predicted using a model of its motion. Associating the set of measurements with the target contained in the map, the robot's state and the feature states are updated simultaneously. This process can diverge very quickly because of the linearisation errors introduced by the EKF and the imperfect data-to-target association usually found in real systems. Furthermore the approach makes use of stereo from motion, meaning that map building information is gathered by moving the vehicle from one position to the next. Moving the vehicle introduces position un-

certainties which in turn affects the accuracy of the stereo information. All these errors quickly accumulate and so the process tends to become unstable.

To improve the stability of the process, Moutarlier et al [7] have described a three stage process, where the system state update is split into two stages. First the robot's position is predicted. After the data-to-target association the robot's state is updated first. The updated state is then used to update the target states. So the simultaneous update of the system state is split into two steps.

Both these approaches rely on computing the cross-covariances between the robot and the targets, and between the targets themselves. This is computationally very expensive and in some cases it is very difficult to obtain the cross-covariances in the first place.

Therefore Leonard et al [6] proposed an approach where the cross-covariances are eliminated. Targets whose trace of their covariance matrix is smaller than a threshold are deemed to be suitable for localisation purposes. Similarly robot positions whose trace of their covariance matrix is also sufficiently small are used for map building purposes. Only the suitable targets and positions are then used in the concurrent localisation and map building process. In order to solve the initialisation problem, they require servo-mounted sensors on the vehicle. This enables them to find features while the vehicle is still stationary, since the stereo from motion problem has now been rid of the odometric uncertainties of the vehicle. Once a feature has been found it can then be used for localisation purposes which ensures accurate position estimates. These in turn enable a consistent map building process and so on.

It may however be undesirable to mount moving sensors on the robot and therefore the stereo from motion problem is still an issue. This paper will describe an approach with fixed sensors which is able to overcome the bootstrap problem and so ensures a stable concurrent localisation and map building process.

In sections 2 and 3 the localisation and map building processes will be looked at separately, while section 4 deals with the combination of both. A short description of the simulation and of the experimental setup are given in sections 5 and 6. Thereafter the obtained results are presented and discussed. The paper ends with a summary and an outlook on future research issues.

2 Localisation

To recap, the localisation process attempts to answer the following question:

Given a set of measurements and a set of features, what is the robot's position?

To answer this question, the robot needs to have a model of its own kinematics (often referred to as the plant model), a model of its sensors (also known as the measurement model) and a model of the observed features. The localisation equations are an implementation of the approach described by Leonard et al with only the most important equations described in appendix A. For a detailed discussion please refer to [6].

Given the models, the robot uses an EKF to update its position estimate as follows:

- Predict the robot's position using the plant model together with information about the robot's translation and rotation which is obtained from the odometry.

- Predict the associated covariance of the position.
- Given the set of features, test which features are visible to which sensor and predict what the measurements should be.
- Compare the predicted measurements to the actual measurements. Only matches that satisfy a certain quality constraint (also called a validation gate) are used for the position update.
- Use the error between the validated measured and predicted measurements to estimate the robot's position.
- Parallel to the position, the associated covariance of the estimate is also determined.

3 Map Building

Map building tries to answer the following question:

Given the robot's position and a set of measurements, what are the sensors "seeing"?

To answer this question, the robot needs to have models of the features that can possibly exist in its environment (in this case planes and corners). The equations to determine the feature parameters from sensor measurements are also an implementation of Leonard et al's approach, with only the most important equations given in appendix B.

It is however not enough to determine what the feature parameters are, since they are calculated using uncertain data such as the robot's position and the sensors' readings. Therefore the associated covariance with these parameters has to be determined. Therefore Leonard's approach was extended to take this into account.

In general a feature is a function of one, even two robot positions and of sensor readings, which can be written as

$$p_t = G(x(k), x(k+i), z(k, k+i)) \quad (6)$$

Using a first order Taylor expansion of G and the assumption that the two robot positions and the measurements are uncorrelated, it can be shown that

$$\begin{aligned} \Lambda_t(k) = & \nabla G_{x(k)} P(k+1/k) \nabla G_{x(k)}^T \\ & + \nabla G_{x(k+i)} P(k+i+1/k+i) \nabla G_{x(k+i)}^T \\ & + \nabla G_{z(k,k+i)} R(k+1, k+i) \nabla G_{z(k,k+i)}^T \end{aligned} \quad (7)$$

where ∇G_x is the Jacobian of function G with respect to x and $R(k+1, k+i)$ is the covariance of the sensor readings. Strictly speaking the independence assumption between the two robot positions is not true, but experience has shown that its violation is not too serious for the whole map building process as a whole.

3.1 Clustering

Since ultrasonic sensors are known to deliver spurious measurements resulting from specular reflections and cross talk, the feature hypotheses generated by the map building process are likely to vary greatly. Therefore the hypotheses need to be clustered to determine mutually consistent feature interpretations. This is principally a data fusion problem.

From the multisensor tracking literature methods exist which effectively cluster hypotheses which originate from the same feature [1]. A new hypothesis is generated which states that the

two hypotheses under comparison have the same origin. This means that the difference between hypothesis $\hat{\mathbf{p}}_i(k)$ and $\hat{\mathbf{p}}_j(k)$ is very small. This hypothesis is accepted when

$$[\hat{\mathbf{p}}_i(k) - \hat{\mathbf{p}}_j(k)]^T [\Lambda_i(k) + \Lambda_j(k)] [\hat{\mathbf{p}}_i(k) - \hat{\mathbf{p}}_j(k)]^T \leq \nu^2 \quad (8)$$

If the hypothesis is accepted, the clustered hypothesis of the two features is given by

$$\hat{\mathbf{p}}_{ij}(k) = \hat{\mathbf{p}}_i + \Lambda_i(\Lambda_i + \Lambda_j)^{-1}(\hat{\mathbf{p}}_j - \hat{\mathbf{p}}_i) \quad (9)$$

$$\Lambda_{ij} = \Lambda_i(\Lambda_i + \Lambda_j)^{-1}\Lambda_j \quad (10)$$

If the cluster is sufficiently large, the feature is deemed to be tentative. From now on the feature is observed to determine whether new observations actually match the new feature. If the new feature is visible it increases the plausibility of the feature. Once a feature has reached a certain plausibility and its uncertainty has decreased sufficiently, it is deemed to be a confirmed target and is then placed into the map for localisation purposes.

4 Concurrent Localisation and Map Building

Now that the two processes of map building and localisation have been described, the problem of how to combine them still remains. At the start of the robot's operation its position is known with very high accuracy, but no map information is available. As the robot moves features are built up and placed into the map. If it takes too long to add a confirmed feature to the map, the danger exists that the robot's positional uncertainty has become too large. Therefore it is important that a confirmed feature be found as soon as possible. The uncertainty threshold for deciding when a tentative feature becomes a confirmed feature is therefore very important. Choosing it too high means that very uncertain features are used for the localisation and so the whole process is bound to diverge. Choosing it too low means that by the time a feature is available, the robot's uncertainty has grown to a point beyond return. From this it becomes clear that during the data-to-target association of the localisation process, the difference between the predicted and actual measurements is not solely due to position uncertainty, but also from the feature uncertainty. Thus while updating the robot's position, the feature's parameters should also be refined at the same time. This is similar to Smith et al and Moutarlier et al, but with the big difference that no explicit cross-covariance matrices are considered.

A predicted sensor measurement can be written as a function of the robot's position and the feature parameters

$$\hat{\mathbf{z}}(k) = h(\hat{\mathbf{x}}(k), \hat{\mathbf{p}}_i) \quad (11)$$

Let $\tilde{\mathbf{z}}(k)$ be the difference between the actual and the predicted measurement (also known as the measurement innovation). The covariance associated with the mismatch is defined as $\mathbf{S}(k)$. Taking the first order Taylor expansion of h and assuming that $\hat{\mathbf{x}}(k)$ and $\hat{\mathbf{p}}_i$ are uncorrelated, it can be shown that the covariance is given by

$$\mathbf{S}(k) = \nabla h_{\hat{\mathbf{x}}(k)} \mathbf{P}(k+1|k) \nabla h_{\hat{\mathbf{x}}(k)}^T + \nabla h_{\hat{\mathbf{p}}_i} \Lambda_i \nabla h_{\hat{\mathbf{p}}_i}^T + \mathbf{R}(k) \quad (12)$$

Similar to the clustering process, a feature is deemed to be suitable for localisation purposes if

$$\tilde{\mathbf{z}}(k)^T \mathbf{S}(k)^{-1} \tilde{\mathbf{z}}(k) \leq \nu^2 \quad (13)$$

In that case the robot position is usually updated as follows

$$\hat{\mathbf{x}}(k+1|k+1) = \hat{\mathbf{x}}(k+1|k) + \mathbf{W}(k)\tilde{\mathbf{z}}(k) \quad (14)$$

where $\hat{\mathbf{x}}(k+1|k)$ is the predicted robot position and $\mathbf{W}(k)$ is the kalman gain. However since the measurement innovation results from *both* the incorrect robot position and the feature parameters, the kalman update equations should reflect this. This requires that the extent to which each of these contribute to the measurement innovation should be estimated. It is determined by alternately setting $\mathbf{P}(k+1|k) = 0$ and $\Lambda_i = 0$ in equation (12), and dividing by $\mathbf{S}(k)$. Furthermore it is assumed that the measurement noise is split equally between the robot position and feature parameters. This yields

$$\beta_{\hat{\mathbf{x}}(k)} = \frac{\nabla h_{\hat{\mathbf{x}}(k)} \mathbf{P}(k+1|k) \nabla h_{\hat{\mathbf{x}}(k)}^T + \frac{\mathbf{R}(k)}{2}}{\mathbf{S}(k)} \quad (15)$$

$$\beta_{\hat{\mathbf{p}}_i} = \frac{\nabla h_{\hat{\mathbf{p}}_i} \Lambda_i \nabla h_{\hat{\mathbf{p}}_i}^T + \frac{\mathbf{R}(k)}{2}}{\mathbf{S}(k)} \quad (16)$$

However since many independence assumptions were made which are not strictly true, the contributions need to be underestimated to ensure stability. Therefore the errors which are used by the EKF to update the robot position and feature parameters, similar to equation (14), are given by

$$\tilde{\mathbf{z}}_{\hat{\mathbf{x}}(k)} = \eta \beta_{\hat{\mathbf{x}}(k)} \tilde{\mathbf{z}}(k) \quad (17)$$

$$\tilde{\mathbf{z}}_{\hat{\mathbf{p}}_i} = \eta \beta_{\hat{\mathbf{p}}_i} \tilde{\mathbf{z}}(k) \quad (18)$$

where $0 \leq \eta \leq 1$.

5 Experimental Setup

To verify the algorithms, they were tested in simulation. The simulated robot contains the same low level pilot functions as our real robot. A realistic ultrasonic sensor model is used for simulating the sensor measurements. The model is capable of dealing with specular reflections, cross talk etc. The robot is equipped with 24 ultrasonic sensors which are arranged cylindrically. The wheel slippage was chosen to reflect the slippage observed on our real robot, namely about 5cm error for every meter of translation and 5 degree error for every 90 degrees of rotation. Furthermore values of $\nu = 2$, $\nu = 6$, and $\eta = 0.7$ were chosen.

6 Results

The robot's task was to follow the outer wall of a 5x5m room (see Fig. 1). The error between the robot's estimated position and orientation and the real position and orientation were recorded. A run without any localisation and map building was performed to act as reference for the performance of the concurrent localisation and map building processes. No a priori map was supplied to the robot. The results of the two runs are shown in Figs. 2-4.

Fig. 2 shows that the absolute position error remains stable during the concurrent localisation and map building run. The absolute angle error (see Fig. 3) however varies greatly. The reason for these large fluctuations is that the position controller with which the robot follows a wall is very unstable during 90 degree turns together with the fact that the linearised plant model becomes inaccurate during large rotations. This can be seen in Figs. 3 and 4 where for example after the manoeuvres during the

interval [250,600], representing turning left twice in the bottom right hand corner and corner above it, the estimated orientation starts to diverge first in one and then in the other direction.

The map which the robot built up during the run is shown in Fig. 5. Its similarity with the room shown in Fig. 1 is reasonably satisfactory. A number of erroneous features still exist in the map because the dynamic deletion of incorrect features was too conservative. It is interesting to note that in the top half of the room corners predominate as features although a line would be the correct representation. This difference is not critical for the accuracy of the localisation process since the corners deliver enough information for the process to remain stable. It does however affect run time performance of the algorithms since they degrade very quickly if the number of features becomes too large. Thus a high level module which is capable of interpreting the contents of the map to deliver a more compact and consistent description is definitely needed.

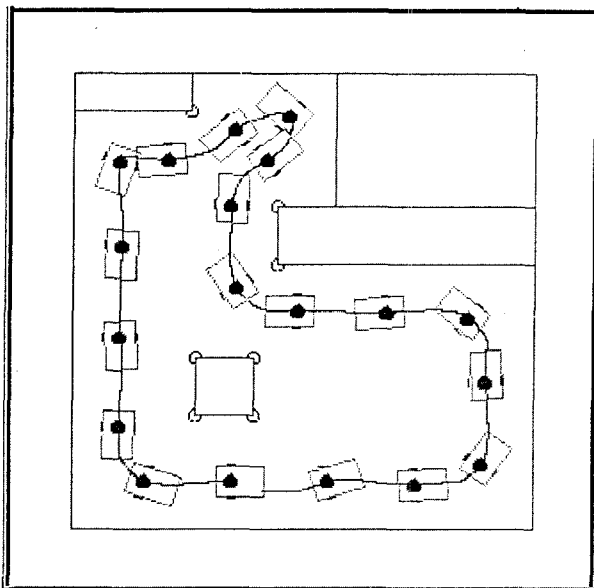


Figure 1: The room within which the robot had to follow the outer walls anti-clockwise. The starting position was chosen under the rectangular obstacle in the lower left half of the room, with the robot facing to the right.

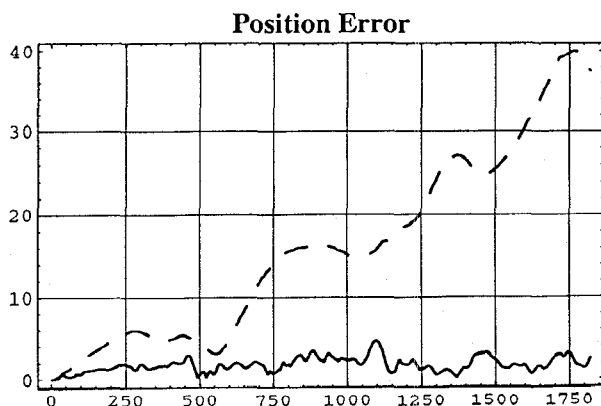


Figure 2: The solid and dashed lines represent the absolute position error in cm of the runs with and without the concurrent localisation and map building respectively. The horizontal axis represents the number of simulation cycles with each cycle representing 50 ms.

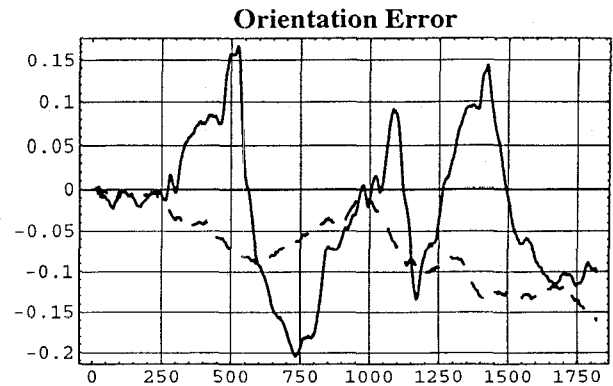


Figure 3: The solid and dashed lines represent the absolute orientation error in radians of the runs with and without the concurrent localisation and map building respectively. The horizontal axis represents the number of simulation cycles with each cycle representing 50 ms.

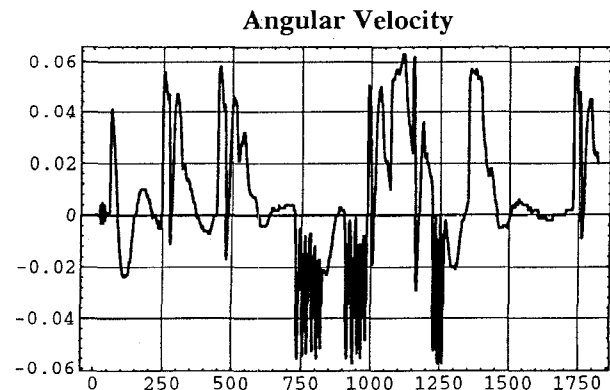


Figure 4: The manoeuvres performed by the robot during the concurrent localisation and map building run. The manoeuvre metric was chosen as the derivative of the robot's orientation in radians. This shows how rough the robot's path is.

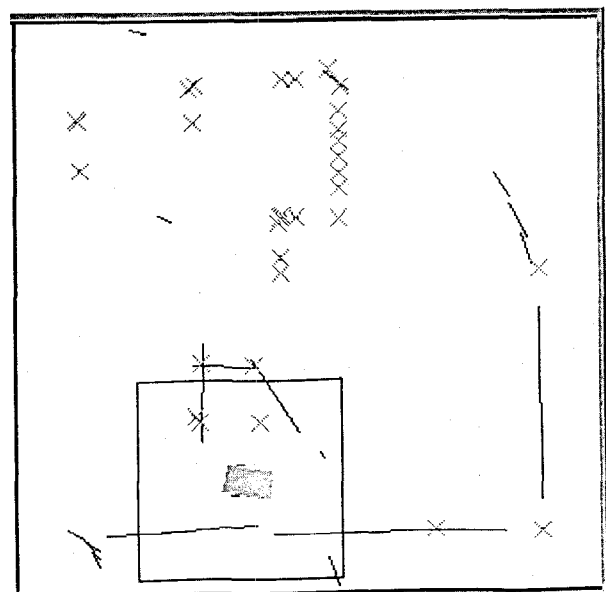


Figure 5: The map generated by the robot during the concurrent localisation and map building run.

7 Conclusions

This paper has presented an approach for dealing with the mobile robot localisation and map building processes simultaneously if the robot is only equipped with fixed sensors. The notorious bootstrapping problem has been overcome by estimating the respective contributions of the position and feature parameter uncertainties to the mismatch between the predicted and actual sensor measurements. The algorithms were tested in simulation and the results showed that the robot navigated stably through its environment. The stability can be further enhanced by using a more accurate plant model which is more robust with respect to excessive robot manoeuvres.

It is envisaged that the algorithms are to be implemented on our mobile robot platform and extensively tested in cluttered environments. Attention will also be given to the real time performance of the algorithms, with special emphasis on the higher level interpretation of the robot's map.

References

- [1] Y. Bar-Shalom and T.E. Fortmann. *Tracking and Data Association*. Academic Press, 1988.
- [2] F. Chevanier and J.L. Crowley. Position estimation for a Mobile Robot Using Vision and Odometry. *Proceedings of the 1992 IEEE Conference on Robotics and Automation*, pages 2588-2593, 1992.
- [3] G. Gilbreath and H.R. Everett. Path planning and collision avoidance for an indoor security robot. *Proceedings SPIE - Int. Soc. Opt. Eng. Mobile Robots III*, 1007:19-27, 1989.
- [4] A.A. Holenstein, M.A. Muller, and E. Badreddin. Mobile Robot Localization in a Structured Environment Cluttered with Obstacles. *Proceedings of the 1992 IEEE Conference on Robotics and Automation*, pages 2576-2581, 1992.
- [5] L. Kleeman. Optimal Estimation of Position and Heading for Mobile Robots Using Ultrasonic Beacons and Dead-Reckoning. *Proceedings of the 1992 IEEE Conference on Robotics and Automation*, pages 2582-2587, 1992.
- [6] J.J. Leonard and H.F. Durrant-Whyte. *Directed Sonar Sensing for Mobile Robot Navigation*. Kluwer Academic Publishers, Boston, London, Dordrecht, 1992.
- [7] P. Moutarlier and Chatila R. Stochastic multisensory data fusion for mobile robot location and environment modeling. In *5th Int. Symposium on Robotics Research*, 1989.
- [8] R. Smith, M. Self, and P. Cheeseman. Estimating uncertain spatial relationships in robotics. In I. Cox and G. Wilfong, editors, *Autonomous Robot Vehicles*. Springer Verlag, 1990.
- [9] A. Zelinsky. Mobile robot map making using sonar. *J. Robot. Syst.*, 8(5):557-577, 1991.

A Localisation Equations

A.1 Plant Model

The following plant model was used:

$$x(k+1) = x(k) + T(k)\cos(\delta(k)) \quad (19)$$

$$y(k+1) = y(k) + T(k)\sin(\delta(k)) \quad (20)$$

$$\theta(k+1) = \theta(k) + \Delta\theta(k) \quad (21)$$

where $T(k)$ is the effective translation, $\Delta\theta(k)$ is the effective rotation of the robot and $\delta(k)$ is the angle between the kinematic centerpoints between time k and $k+1$ (see Fig. 6).

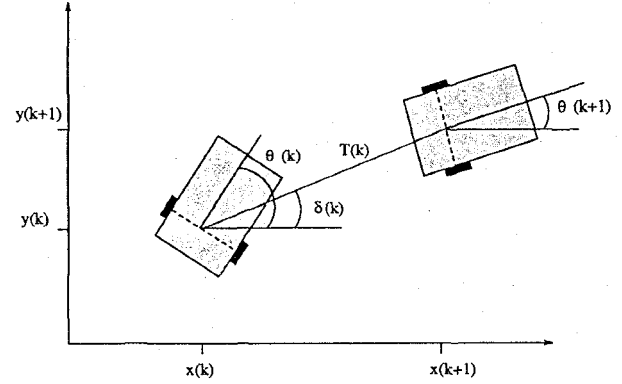


Figure 6: The robot plant model.

A.2 Feature Models

The map consists of two types of features, namely planes and corners. In a 2-D approximation these reduce to lines and points respectively. A plane's state is represented by a finite segment of an infinite line in its hessian normal form, or $p_p = (p_v, p_r, p_\theta, d_l, d_r)$ (see Fig. 7). A corner's state is represented by the global cartesian coordinates of a point, or $p_c = (x, y)$.

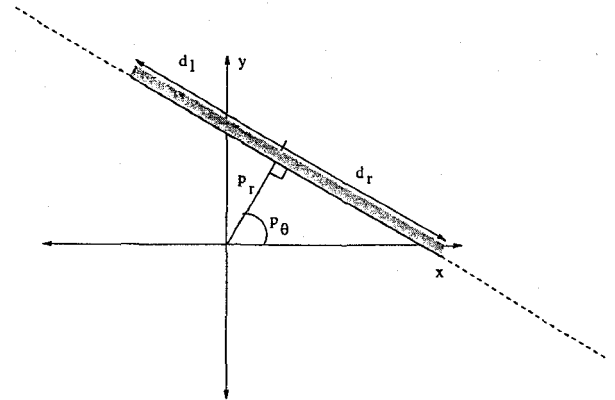


Figure 7: The hessian normal form of a finite line segment visible from one side only.

A.3 Measurement model

The robot's measurement model describes what reading a sensor should have, given the robot's state, the observed feature and the sensor's position relative to the robot. The absolute sensor position is given by

$$x_s(k) = x(k) + x'_s\cos(\theta(k)) - y'_s\sin(\theta(k)) \quad (22)$$

$$y_s(k) = y(k) + x'_s\sin(\theta(k)) + y'_s\cos(\theta(k)) \quad (23)$$

$$\alpha_s(k) = \theta(k) + \alpha'_s(k) \quad (24)$$

where x'_s, y'_s and α'_s are the relative position of the sensor with respect to the robot (see Fig. 8).

In this approach the robot is equipped with ultrasonic sensors which are plagued by poor angular resolution. Therefore the

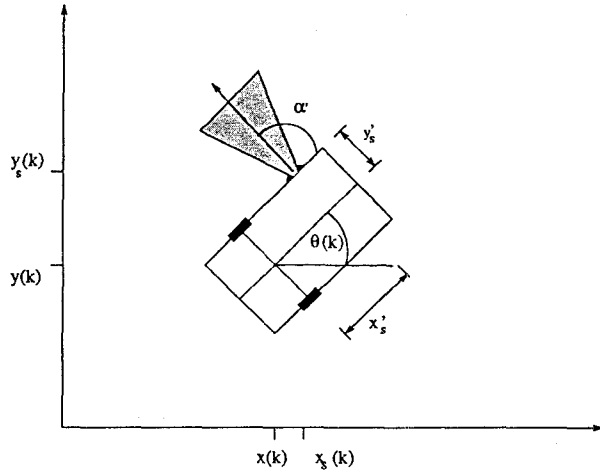


Figure 8: The relationship between a sensor and the state of the robot.

predicted sensor reading only considers the range information. For the two types of features the predicted measurement is given by

$$\hat{z}_c(k) = [(p_x - x(k) - x'_s \cos(\theta(k)) + y'_s \sin(\theta(k)))^2 + (p_y - y(k) - x'_s \sin(\theta(k)) - y'_s \cos(\theta(k)))^2]^{\frac{1}{2}} \quad (25)$$

$$\hat{z}_p(k) = p_v [p_r - (x(k) + x'_s \cos(\theta(k)) - y'_s \sin(\theta(k))) \cos(p_\theta) - (y(k) + x'_s \sin(\theta(k)) + y'_s \cos(\theta(k))) \sin(p_\theta)] \quad (26)$$

The angular information of the sensor is used to test whether a feature is visible to the sensor. If the visibility test fails no predicted reading is generated.

B Map Building Equations

Each range measurement describes a circle with the sensor at its origin and the radius equal to the measurement. To be able to determine what type of feature the sensor is observing, the robot needs another measurement. Constructing a local coordinate frame with the position of sensor 1 at the origin and the position of sensor 2 at distance d on the x -axis, the two circles originating from the measurements are used to calculate the features parameters (see Fig. 9). For planes it can be shown that they lie tangent to the circles, and the corners lie at the intersection of the circles.

In general the relationship between the two absolute sensor positions is given by

$$d(k) = [(y_s(k+i) - y_s(k))^2 + (x_s(k+i) - x_s(k))^2]^{\frac{1}{2}} \quad (27)$$

$$\tan(\gamma) = \frac{y_s(k+i) - y_s(k)}{x_s(k+i) - x_s(k)} \quad (28)$$

$$(29)$$

The corner parameters are then given by

$$\cos(\phi) = \frac{z(k)^2 - z(k+i)^2 + d^2}{2d(k)z(k)} \quad (30)$$

$$x_c = z(k) \cos(\phi) \quad (31)$$

$$y_c = z(k) \sin(\phi) \quad (32)$$

$$x_g = x_c \cos(\gamma) - y_c \sin(\gamma) + x_s(k) \quad (33)$$

$$y_g = x_c \sin(\gamma) + y_c \cos(\gamma) + y_s(k) \quad (34)$$

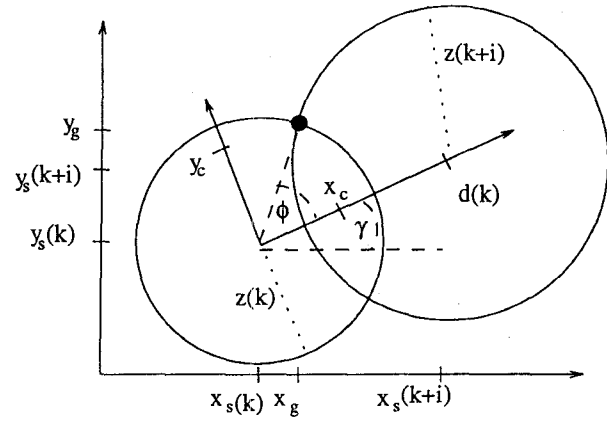


Figure 9: The construction of a corner feature from two sensor measurements.

The plane parameters are then given by

$$\cos(\phi) = \frac{z(k) - z(k+i)}{d(k)} \quad (35)$$

$$\theta_p = \phi + \gamma \quad (36)$$

$$r_p = x_g \cos(\theta_p) + y_g \sin(\theta_p) \quad (37)$$

The other plane parameters such as p_v , d_l and d_r are determined by considering from which direction the sensors observed the plane, and where the tangent contact points lie on the infinite plane.